



230968336

SAMEER RAJ

WEEK-6

Exercise 1: Hill-Climbing Search for Feature Selection in Predictive Modeling

```
In [1]: import numpy as np
import random
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

data = load_iris()
X = data.data
y = data.target
n_features = X.shape[1]

def evaluate(state):
    if sum(state) == 0:
        return 0

    selected_features = [i for i in range(len(state)) if state[i] == 1]
    X_subset = X[:, selected_features]

    model = LogisticRegression(max_iter=200)
    scores = cross_val_score(model, X_subset, y, cv=5)
    return scores.mean()

def generate_neighbors(state):
    neighbors = []
    for i in range(len(state)):
        neighbor = state.copy()
        neighbor[i] = 1 - neighbor[i]
        neighbors.append(neighbor)
    return neighbors

def hill_climbing():
    current_state = [random.choice([0, 1]) for _ in range(n_features)]
    current_score = evaluate(current_state)

    while True:
        neighbors = generate_neighbors(current_state)
        neighbor_scores = [(neighbor, evaluate(neighbor)) for neighbor in neighbors]

        best_neighbor, best_score = max(neighbor_scores, key=lambda x: x[1])
```

```

        if best_score <= current_score:
            break

        current_state = best_neighbor
        current_score = best_score

    return current_state, current_score

if __name__ == "__main__":
    runs = 5
    for i in range(runs):
        best_state, best_score = hill_climbing()
        print(f"Run {i+1}:")
        print("Selected Features:", best_state)
        print("Accuracy:", round(best_score, 4))
        print("-" * 40)

```

```

Run 1:
Selected Features: [1, 1, 1, 1]
Accuracy: 0.9733
-----
Run 2:
Selected Features: [0, 0, 0, 1]
Accuracy: 0.96
-----
Run 3:
Selected Features: [0, 0, 0, 1]
Accuracy: 0.96
-----
Run 4:
Selected Features: [1, 1, 1, 1]
Accuracy: 0.9733
-----
Run 5:
Selected Features: [0, 0, 0, 1]
Accuracy: 0.96
-----

```

Exercise 2: Simulated Annealing for Vehicle Routing Optimization

```

In [2]: import numpy as np
import random
import matplotlib.pyplot as plt

n_cities = 15

np.random.seed(42)
cities = np.random.rand(n_cities, 2) * 100

def total_distance(route, cities):
    distance = 0
    for i in range(len(route)):
        from_city = cities[route[i]]

```

```

        to_city = cities[route[(i + 1) % len(route)]]
        distance += np.linalg.norm(from_city - to_city)
    return distance

def get_neighbor(route):
    new_route = route.copy()
    i, j = random.sample(range(len(route)), 2)
    new_route[i], new_route[j] = new_route[j], new_route[i]
    return new_route

def simulated_annealing(cities, initial_temp=1000, cooling_rate=0.995, min_tem

    current_route = list(range(len(cities)))
    random.shuffle(current_route)

    current_cost = total_distance(current_route, cities)

    best_route = current_route.copy()
    best_cost = current_cost

    T = initial_temp

    while T > min_temp:

        neighbor = get_neighbor(current_route)
        neighbor_cost = total_distance(neighbor, cities)

        deltaE = neighbor_cost - current_cost

        if deltaE < 0:
            current_route = neighbor
            current_cost = neighbor_cost
        else:
            probability = np.exp(-deltaE / T)
            if random.random() < probability:
                current_route = neighbor
                current_cost = neighbor_cost

        if current_cost < best_cost:
            best_route = current_route.copy()
            best_cost = current_cost

        T *= cooling_rate

    return best_route, best_cost

best_route, best_cost = simulated_annealing(cities)

print("Best Route:", best_route)
print("Best Cost:", round(best_cost, 2))

```

Best Route: [6, 10, 14, 9, 2, 7, 11, 8, 13, 3, 5, 0, 12, 4, 1]
 Best Cost: 320.53

Exercise 3 (Revised): Genetic Algorithm for Autonomous Drone Path Planning

```
In [4]: import numpy as np
import random
import matplotlib.pyplot as plt

MAP_SIZE = 100

start = np.array([5, 5])
end = np.array([95, 95])

obstacles = [
    (40, 40, 10),
    (70, 60, 8),
    (50, 80, 7)
]

POP_SIZE = 50
WAYPOINTS = 6
GENERATIONS = 150
MUTATION_RATE = 0.2
TOURNAMENT_SIZE = 3

def path_length(path):
    total = 0
    for i in range(len(path) - 1):
        total += np.linalg.norm(path[i] - path[i+1])
    return total

def point_in_obstacle(point):
    for (ox, oy, r) in obstacles:
        if np.linalg.norm(point - np.array([ox, oy])) <= r:
            return True
    return False

def fitness(chromosome):
    path = [start] + chromosome + [end]
    path = np.array(path)

    length = path_length(path)

    penalty = 0
    for point in chromosome:
        if point_in_obstacle(point):
            penalty += 1000

    return length + penalty

def create_random_chromosome():
    return [np.array([random.uniform(0, MAP_SIZE),
                    random.uniform(0, MAP_SIZE)])
            for _ in range(WAYPOINTS)]
```

```

def initialize_population():
    return [create_random_chromosome() for _ in range(POP_SIZE)]

def tournament_selection(population):
    selected = random.sample(population, TOURNAMENT_SIZE)
    selected.sort(key=lambda c: fitness(c))
    return selected[0]

def crossover(parent1, parent2):
    point = random.randint(1, WAYPOINTS - 1)
    child = parent1[:point] + parent2[point:]
    return child

def mutate(chromosome):
    for i in range(WAYPOINTS):
        if random.random() < MUTATION_RATE:
            chromosome[i] += np.random.normal(0, 5, 2)

            chromosome[i] = np.clip(chromosome[i], 0, MAP_SIZE)

    return chromosome

def genetic_algorithm():
    population = initialize_population()

    best_solution = None
    best_fitness = float('inf')

    for generation in range(GENERATIONS):
        new_population = []

        for _ in range(POP_SIZE):
            parent1 = tournament_selection(population)
            parent2 = tournament_selection(population)

            child = crossover(parent1, parent2)
            child = mutate(child)

            new_population.append(child)

        population = new_population

        for individual in population:
            f = fitness(individual)
            if f < best_fitness:
                best_fitness = f
                best_solution = individual

    if generation % 20 == 0:
        print(f"Generation {generation} | Best Fitness: {best_fitness:.2f}")

```

```
    return best_solution, best_fitness

best_path, best_cost = genetic_algorithm()

print("\nFinal Best Cost:", round(best_cost, 2))
```

```
Generation 0 | Best Fitness: 232.86
Generation 20 | Best Fitness: 151.65
Generation 40 | Best Fitness: 144.38
Generation 60 | Best Fitness: 138.65
Generation 80 | Best Fitness: 138.47
Generation 100 | Best Fitness: 135.14
Generation 120 | Best Fitness: 135.14
Generation 140 | Best Fitness: 132.86
```

Final Best Cost: 132.86